Supplementary Material for RevTrace: A Transactional Runtime for Feedback-State Integrity in Revision-Generative ASR Serving

Bin Zhou, Jiyuan Miao, and Xi Cao

S1. Protocol Record and Validation Fields

This supplement expands the field-level protocol and proofs summarized in Section III of the main paper. Committed state refers only to the feedback sub-state H written from generated output.

A. Transaction and certificate records

For proposal or authority output Y_t, the runtime materializes

$$\tau_t = (\mathrm{sid}, e, p, v_{\mathrm{parent}}, [a_s, a_e), Y_t, \Phi(Y_t; H_t), \mathrm{cert}_t).$$

The fields carry the session identifier, contract epoch, active policy, parent version read, covered audio interval, realized output, projected feedback state, and certificate status. Proposal and authority granularities g_p and g_a define $\rho = g_a/g_p \geq 1$.

For an authority result $\Phi_j^a = \Phi(Y_j^a; H_{v_j})$, the protected coordinator issues

$$\mathrm{cert}_j = \mathrm{Bind}(\mathrm{sid}_j, e_j, p_j, v_j, [f_j, f_{j+1}), h(A_{[f_j, f_{j+1})}), h(\Phi_j^a), \chi_j).$$

The configuration digest χ_j covers the model checkpoint, tokenizer, decoding parameters, projection-adapter version, numerical precision, authority-runtime version, and the randomness policy required by the contract. The audio and projection digests are recomputed over immutable bytes. Each certificate also carries an unconsumed/consumed state maintained atomically by the ledger.

B. Atomic validation

For

$$\Gamma_j = (\mathrm{sid}_j, e_j, p_j, v_j, [f_j, f_{j+1}), \Phi_j^a, \mathrm{cert}_j),$$

validation and installation execute as one compare-and-swap transaction. The runtime checks:

1) *session fence*: the session identifier denotes the live ledger;
2) *contract fence*: epoch and policy equal the active contract;
3) *parent ownership*: v_j equals the committed state version;
4) *frontier continuity*: the interval begins at the committed frontier and advances it without a gap or overlap;
5) *content binding*: all certificate fields match the presented tuple, and the audio and projection digests recompute;
6) *single use*: the certificate has not been consumed.

Failure of any field rejects the tuple without changing H, the frontier, or the certificate-consumption bit. Success installs Φ_j^a, advances the frontier and version, and consumes cert_j atomically.

S2. Detailed Safety Proofs

Theorem S2.1 (Uncertified-Ancestry Exclusion)**.** *If committed state changes exclusively through valid commits, no fine projection unbound by a valid certificate can enter the ancestry of any future committed state.*

Proof. Index accepted commits by j. At initialization, H_0 is contract-defined and contains no generated ancestor. Assume every generated projection on the unique path to H_j is certificate-bound. The only operation that can create H_{j+1} is an accepted compare-and-swap. Content validation recomputes $h(\Phi_j^a)$ and requires it to match the projection digest in the presented certificate; hence different content cannot be installed. A displayed fine projection has no commit capability and therefore cannot alter the path.

If a fine projection differs from Φ_j^a, it fails content binding. If it happens to be bitwise equal, the installed bytes are nevertheless authorized by the authority certificate and are attributable to that valid commit, not to the uncertified proposal. Thus every generated ancestor of H_{j+1} is certificate-bound, completing the induction. □

Theorem S2.2 (Fork-Free Committed-State Linearity)**.** *If validation and installation form one atomic compare-and-swap and each certificate is consumed at most once, parent-version and frontier checks make accepted commits fork-free, gap-free, and overlap-free.*

Proof. Suppose two distinct candidates read parent version v. At most one can win the compare-and-swap that both validates and installs against v. The winner increments the committed version, so the loser is stale when it reaches the serialization point and fails the parent check. Therefore no two accepted children share one committed parent. Single-use consumption additionally prevents a winning certificate from being replayed at another version.

Let the current committed frontier be f_j. A winning interval must begin at f_j and advances the frontier to f_{j+1}. The next accepted interval must start at this new value. Induction over winners therefore yields the single sequence $[f_0, f_1), [f_1, f_2), \ldots$, excluding gaps and overlaps. Combining unique parentage with contiguous intervals yields one linear committed history. □

Corollary S2.3 (Stale-descendant rejection)**.** *A descendant computed from $v_{\mathrm{parent}} \neq \mathrm{version}(H_j)$ cannot commit, even if its output or projection equals current content.*

Proof. Parent ownership is a required field check independent of content equality; the mismatch rejects the compare-and-swap before installation. □

Remark S2.4 (Concurrency boundary). The compare-and-swap is a serialization point for accepted commits. The claim does not extend to Herlihy–Wing linearizability of all client invocations and responses, for which a separate real-time concurrent-object specification would be required.

Theorem S2.5 (Granularity-Independent Safety)**.** *For every $\rho \geq 1$, Theorems S2.1 and S2.2 hold if each authority segment starts at the committed parent, covers the next contiguous frontier, produces a legal projection, and passes certificate and version checks.*

Proof. Neither preceding proof counts proposal windows or constrains segment length. The ancestry proof uses only content binding and the exclusive commit path. The linear-history proof uses only atomic parent ownership, single-use consumption, and frontier continuity. Increasing ρ changes how many fine regions a certificate covers but changes none of these predicates. Therefore both invariants are preserved. □

At $\rho = 1$, the proposal and authority regions coincide, directional coverage is the identity witness, and selective fine replay reduces to risk-triggered re-verification. This is a protocol reduction only: the evaluated SG and Coarse/Hybrid contracts use distinct FP16-singleton and FP32-batched canonical paths and need not be bitwise equal.

S3. Authority-Cost Assumptions

Let T_p be proposer time, B concurrent sessions, q the SG verification fraction, r the Hybrid replay fraction, C_s singleton authority cost per fine region, $C_c(\rho, B)$ batched coarse cost, Q_s singleton queueing/fragmentation, and C_{commit} transaction overhead. The main-paper decomposition assumes

$$T_{\text{service}} \approx \max(T_p, T^a) + T_{\text{repair}}.$$

Without assuming either lane is critical, Coarse-DGFT improves service time whenever

$$\begin{aligned}&\max(T_p, C_c(\rho, B) + rB\rho C_s + C_{\text{commit}}) + T^D_{\text{repair}} \\ &\quad < \max(T_p, qB\rho C_s + Q_s) + T^S_{\text{repair}}.\end{aligned}$$

The simpler inequality stated in the main paper is sufficient only when the authority lane is on or near the critical path in both configurations and repair costs are comparable. These equations predict operating-point behavior; they do not identify a precision-independent causal effect because the evaluated contracts bind different complete authority paths.

S4. Host-Side Protocol Overhead

We isolate CPU transaction cost with the production functions in `src/revtrace/dgft.py`. Each of 31 randomized-order repeats times 20,000 valid calls after 2,000 warmup calls per variant. The fixed fixture uses a 256-token parent prompt, 32 display tokens, and eight speculative descendants. Table I reports percentiles across the 31 repeat-level per-operation aggregates. The host runs Windows 11 and CPython 3.13.5 on a 24-logical-CPU Intel Family 6 Model 183 processor; the review artifact records the complete environment and source hashes.

TABLE I
Host-side production-function microbenchmark (μs/operation).

Path	Median	P95	P99
State materialization; no validation	19.077	19.813	19.885
Job/owner/version/frontier validation	24.835	25.636	25.887
Job plus full certificate validation	80.604	82.902	85.866
Production commit	101.107	104.486	104.514

The paired median increment from no-check materialization to the complete production commit is 81.960 μs. In 1,048 existing online commits, the broader coordinator critical section has median/P95/P99 1.318/2.717/3.784 ms; that interval also includes lock wait, conversion, certificate construction and hashing, materialization, and pending-version updates, so it is only a deployed-path upper envelope. Neither measurement is used as GPU or end-to-end speedup evidence.